

Introduction to Deep Learning

Magda Gregorová

magda.gregorova@thws.de

March 2026

Licensed according to [CC-BY-SA 4.0](#)

Contents

I. Foundations	3
1. What is Deep Learning?	3
1.1. Demonstration	3
1.2. What is Deep Learning?	3
1.3. Brief History	4
1.4. Why Now?	6
1.4.1. Data	6
1.4.2. Compute	7
1.4.3. Algorithms	7
1.4.4. Community	8
1.5. What Can Deep Learning Do?	9
1.5.1. Language and Communication	9
1.5.2. Vision and Perception	9
1.5.3. Healthcare and Medicine	9
1.5.4. Generation	9
1.5.5. Scientific Discovery	10
1.5.6. Decision Making and Control	10
2. CNNs in Practice	10
2.1. Transfer Learning	10
2.1.1. Three Regimes	10
2.1.2. Why Pretrained Models Work	11
2.1.3. Feature Extraction vs Fine-tuning	11
2.1.4. When to Use Which	12
2.1.5. Practical Tuning	12
2.2. Practical CNN Training	13
2.2.1. Sources of Pretrained Models	13
2.2.2. Adapting a Pretrained Model	13
2.2.3. Image Transforms Pipeline	14

2.3. Beyond Classification	15
2.3.1. Object Detection	15
2.3.2. Semantic Segmentation	16
2.3.3. Instance Segmentation	16
2.3.4. Annotation Cost	17
Acknowledgements	17

Part I.

Foundations

1. What is Deep Learning?

1.1. Demonstration

► Session 1, Slides
1-12

Before any theory, consider the following experiment. A large language model - specifically Claude - is prompted with a single sentence:

“Write me a PyTorch script that trains a neural network to classify CIFAR-10 images into 10 classes. Use a simple MLP architecture. Print the training loss each epoch and plot a few example predictions with their true and predicted labels at the end.”

The model produces a complete, runnable Python script. When executed, the script downloads the CIFAR-10 dataset - 60,000 colour images of 32×32 pixels across 10 classes - trains a neural network, and achieves reasonable classification accuracy. A second prompt requesting a CNN instead of an MLP produces a different script that achieves noticeably higher accuracy on the same task.

This demonstration works. It is genuinely easy. And therein lies the problem: *what just happened?*

The script contains weights, layers, a loss function, a training loop, gradient updates. None of these were explained. The goal of this course is to open that black box - not to prompt better, but to think deeper.

1.2. What is Deep Learning?

Artificial intelligence, machine learning, and deep learning are often used interchangeably in popular discourse, but they refer to distinct and nested concepts.

► Session 1: What
is Deep Learning?

Artificial intelligence is the broadest term - it encompasses any technique that enables machines to exhibit behaviour we would consider intelligent, including rule-based expert systems, search algorithms, and learned models.

Machine learning is a subfield of AI in which systems learn from data rather than being programmed with explicit rules. Instead of writing down every decision, we provide examples and let the system discover the underlying patterns.

Deep learning is a subfield of machine learning that uses neural networks with many layers - so-called deep neural networks. The depth allows the network to learn increasingly abstract representations of the input data.

Generative AI - encompassing large language models as well as image, video, and music generation systems - is a specific and currently prominent subset of deep learning. What the general public refers to as “AI” today is almost exclusively generative AI, which sits at the innermost level of this hierarchy.

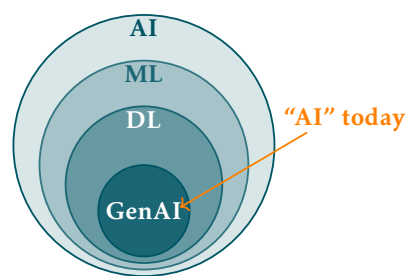


Figure 1.1: The relationship between AI, machine learning, deep learning, and generative AI.

Definition 1 (Deep Learning). Deep learning is a class of machine learning methods that use multi-layered neural networks to learn hierarchical representations directly from raw data.

This definition will sharpen as the course progresses. For now it is sufficient to hold it loosely - by the end of the course every word in it will have a concrete meaning.

1.3. Brief History

Figure 1.2 shows the key milestones in the development of deep learning alongside the periods of reduced funding and interest known as AI winters. For a comprehensive overview of the field's history see [lecun2015deeplearning](#) and [goodfellow2016deeplearning](#).

► Session 1: A Brief History of Deep Learning

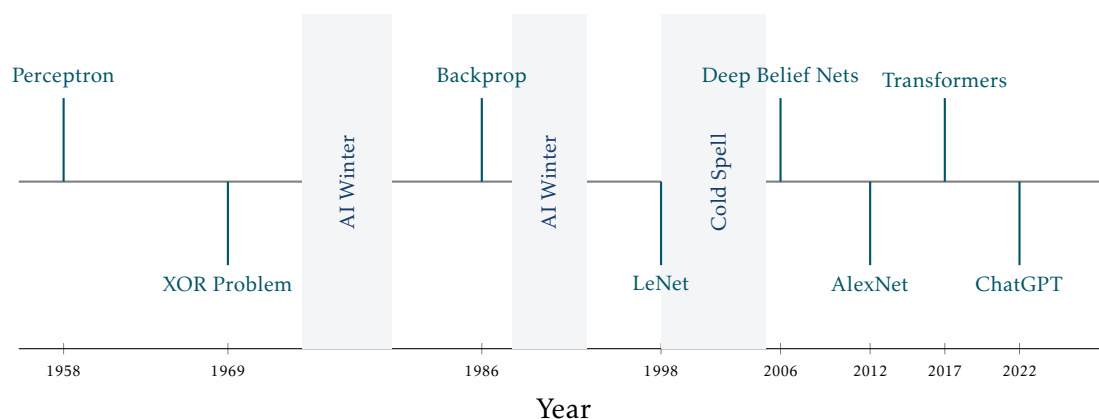


Figure 1.2: Key milestones in the history of deep learning. Shaded regions indicate AI winters and the cold spell of the early 2000s.

1958 - Perceptron. Frank **rosenblatt1958perceptron** introduced the perceptron - the first trainable neural network. It could learn to classify linearly separable patterns by adjusting weighted connections between inputs and a single output unit. The result generated enormous excitement and equally enormous overpromising about the future of machine intelligence. Rosenblatt himself claimed the perceptron would eventually be able to walk, talk, see, write, reproduce itself, and be conscious of its existence - a level of optimism that would later contribute to the severity of the backlash.

The years following the perceptron saw genuine progress. The ADALINE model **widrow1960adaline** introduced the least mean squares learning rule, which remains in use today. The concept of multilayer networks was understood theoretically, and researchers were actively exploring how to train them. There was a widespread belief that human-level artificial intelligence was only a decade away - a belief shared by many prominent scientists and echoed in generous research funding from DARPA and other agencies.

However, the limitations of single-layer networks were becoming apparent in practice. Many of the problems researchers wanted to solve - pattern recognition, language understanding, game playing - turned out to be far harder than anticipated. The gap between the promise of the perceptron and the reality of what it could achieve was widening.

1969 - XOR Problem. **minsky1969perceptrons** published *Perceptrons*, demonstrating formally that single-layer networks could not solve problems as simple as the XOR function. The 1970s were a difficult period for neural network research. The Lighthill

Report **lighthill1973** delivered a damning assessment of AI research in 1973, concluding that the field had failed to deliver on its promises. DARPA cut funding significantly and many researchers moved away from neural networks entirely. This was the first AI winter.

Yet work continued quietly. Paul Werbos derived the backpropagation algorithm in his 1974 PhD thesis **werbos1974thesis** - years before it became widely known. **fukushima1980neocognitron** introduced the Neocognitron, a hierarchical neural network inspired by the visual cortex that anticipated many ideas later formalised in convolutional networks. **hopfield1982neural** networks demonstrated that recurrent networks could function as associative memories. These contributions kept the field alive through the cold years, largely unnoticed by the mainstream AI community which had shifted its focus toward symbolic reasoning and expert systems.

Japan's role in this period deserves particular mention. Fukushima's Neocognitron emerged from NHK Science and Technology Research Laboratories in Tokyo. In 1982 the Japanese government launched the ambitious Fifth Generation Computer Project **fgcs1982**, investing heavily in AI hardware and parallel computing. The project caused significant alarm in the US and UK and triggered substantial counter-investment in AI research on both sides of the Atlantic. Its ultimate failure to deliver human-level reasoning contributed to the disillusionment of the second AI winter.

1986 - Backpropagation. **rumelhart1986backprop** popularised the backpropagation algorithm, providing a practical method for training multi-layer networks by propagating error gradients backwards through the layers. The revival of backpropagation generated enormous enthusiasm. Multilayer networks were successfully applied to speech recognition, image classification, and financial forecasting. LeCun applied backpropagation to convolutional networks at Bell Labs, demonstrating practical handwriting recognition systems used by the US postal service. Hochreiter and Schmidhuber introduced the Long Short-Term Memory (LSTM) network in 1997 **hochreiter1997lstm**, solving the vanishing gradient problem for recurrent networks and laying the foundation for sequence modelling over the following two decades.

However, the second AI winter arrived in the late 1980s and early 1990s. Expert systems - rule-based AI programs that had attracted enormous commercial investment - collapsed as their maintenance costs proved prohibitive and their brittleness became apparent. The Lisp machine market crashed in 1987. DARPA again cut funding. Neural networks hit their own practical ceiling: training deep networks remained unstable, datasets were small, and support vector machines (SVMs) - introduced by **vapnik1995svm** - offered better performance on many tasks with stronger theoretical guarantees. By the mid-1990s SVMs had largely displaced neural networks as the method of choice for classification problems.

1998 - LeNet. Yann **lecun1998gradient** demonstrated convolutional neural networks for handwritten digit recognition, achieving state-of-the-art results on the MNIST dataset. Despite its success, neural networks remained out of favour through the early 2000s - a period sometimes called the cold spell - as support vector machines and other kernel methods dominated the field.

This cold spell was not a complete winter - small groups continued working on deep architectures. **bengio2004representation** explored the difficulty of training deep networks, identifying the vanishing gradient problem as the central obstacle. Hinton's group at Toronto worked on unsupervised pretraining as a way to initialise deep networks before fine-tuning with backpropagation. The theoretical groundwork for the

2006 breakthrough was being laid quietly throughout this period.

2006 - Deep Belief Nets. [hinton2006belief](#) showed that deep networks could be trained effectively using a layer-wise pretraining procedure. This reignited interest in deep architectures and marked the beginning of the modern deep learning era, even before large datasets and GPU training were fully established.

The key developments of this period were infrastructural rather than algorithmic. [raina2009gpu](#) demonstrated that GPUs could accelerate neural network training by an order of magnitude, making experiments that previously took weeks feasible in hours. The ImageNet dataset was released by [deng2009imagenet](#), providing for the first time a benchmark large and challenging enough to reveal the true potential of deep architectures. [nair2010relu](#) introduced the ReLU activation function, which proved dramatically more effective than sigmoid and tanh activations for training deep networks. These three developments - GPU training, large-scale data, and better activations - set the stage for AlexNet's breakthrough in 2012.

2012 - AlexNet. [krizhevsky2012alexnet](#) entered a deep convolutional network trained on GPUs into the ImageNet challenge and won by a margin that shocked the field - reducing the top-5 error rate from 26% to 15% in a single year. This moment is widely regarded as the turning point that established deep learning as the dominant paradigm in machine learning.

2017 - Transformers. [vaswani2017attention](#) introduced the transformer architecture in the paper *Attention Is All You Need*, replacing recurrent computation with self-attention mechanisms. The transformer became the foundation of virtually all modern large-scale models in both language and vision.

2022 - ChatGPT. OpenAI released ChatGPT, bringing large language models to a mass audience for the first time. Within two months it had reached 100 million users, making it the fastest-growing consumer application in history and triggering a wave of public and institutional interest in artificial intelligence that continues today.

The pattern visible in Figure 1.2 is worth noting: the history of deep learning is one of recurring cycles of excitement and disappointment, punctuated by genuine breakthroughs. The current era is distinguished not just by the scale of the results but by the breadth of their impact - for the first time, deep learning systems are changing how ordinary people work and communicate on a daily basis.

1.4. Why Now?

Deep learning is not a new idea - neural networks have existed since the 1950s. What changed is the confluence of four factors that together made the approach viable at scale. None of these alone was sufficient; together they triggered a revolution.

► Session 1: Why Now? - Four Driving Forces

1.4.1. Data

The internet created the largest dataset in history by accident. Every uploaded image, every written caption, every search query became potential training material. This was crystallised deliberately in benchmark datasets - ImageNet (2009) provided 1.2 million labelled images across 1,000 categories, and GLUE provided a standardised benchmark for natural language understanding. These benchmarks did not just measure progress, they created it by giving the community a shared target to optimise against.

► Session 1: Why Now? - Four Driving Forces, Data - Scale Revolution

Crucially, the field also learned that labels are not always necessary. Self-supervised and weakly supervised learning techniques extract signal directly from unlabelled data,

massively expanding what counts as usable training material. The web, in its entirety, became a dataset.

Figure 1.3 shows the growth in dataset scale over time. Two aspects deserve attention. First, the left axis is logarithmic - each gridline represents a tenfold increase, so the jump from ImageNet to LAION-5B represents roughly four thousand times more data. Second, the orange line shows that resolution grew alongside quantity - from 28×28 pixels for MNIST to 1024×1024 for LAION-5B, a 1,000-fold increase in pixels per image. The two dimensions of scale - how many images and how much information per image - grew together.

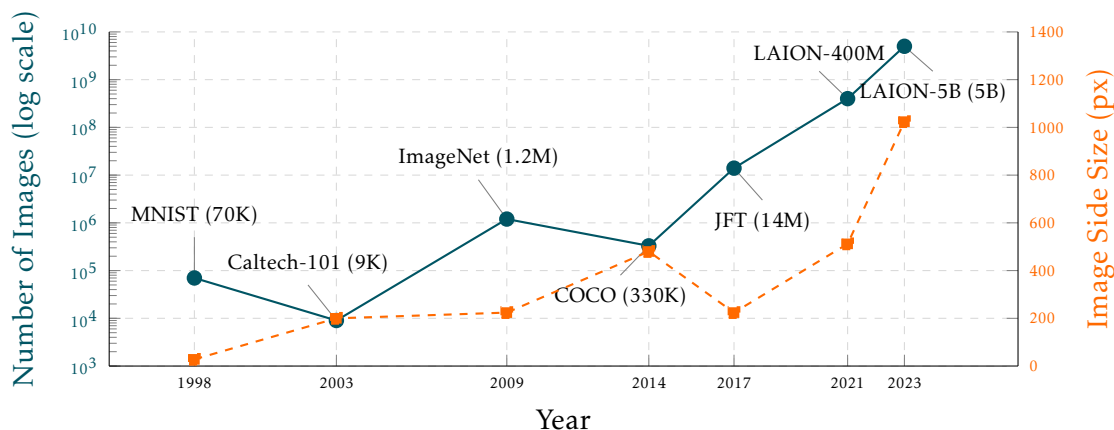


Figure 1.3: Image dataset sizes and resolutions over time. Blue line: number of images (log scale, left axis). Orange line: image side size in pixels (linear scale, right axis).

1.4.2. Compute

Graphics processing units, designed for the parallel computation of gaming graphics, turned out to be ideally suited for the matrix multiplications at the heart of neural network training. NVIDIA's CUDA programming model made GPUs accessible to researchers without hardware expertise. Custom chips - Google's TPUs, Apple Silicon, and a wave of AI-specific inference hardware - followed, with architectures designed around deep learning from the ground up.

Cloud platforms such as AWS, Microsoft Azure, and Google Cloud democratised access to large-scale compute - a researcher can now rent state-of-the-art hardware by the hour. Learning platforms such as Google Colab, Kaggle Notebooks, and Lightning.ai went further, providing free GPU access directly in the browser with zero setup. As Figure 1.4 shows, GPU performance has grown from single-digit TFLOPS in 2012 to nearly 2,000 TFLOPS in 2024.

1.4.3. Algorithms

Several targeted innovations removed practical obstacles that had blocked progress for decades. At the architectural level, the multilayer perceptron (MLP), convolutional neural network (CNN), and recurrent neural network (RNN) established the foundational building blocks, all trained via the backpropagation algorithm. The ReLU activation function mitigated the vanishing gradient problem that had plagued deep networks. Batch normalisation stabilised training. Dropout provided effective regularisation.

► Session 1:
Compute &
Progress

► Session 1: Why
Now? - Four
Driving Forces,
Compute &
Progress

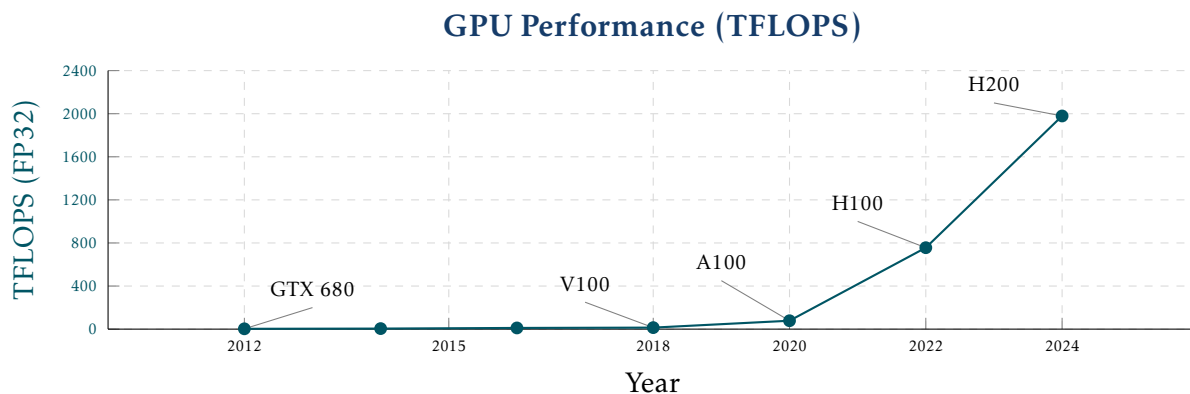


Figure 1.4: GPU performance in TFLOPS (FP32) from 2012 to 2024.

At a higher level, residual connections (ResNets) made it possible to train networks of hundreds of layers. The attention mechanism and the transformer architecture, introduced in 2017, replaced recurrent computation entirely and became the foundation of modern language and vision models.

Figure 1.5 illustrates the impact of these algorithmic advances on the ImageNet benchmark. In 2011, the best classical methods using SVMs achieved around 74% top-5 accuracy. AlexNet's jump to 84.7% in 2012 marked the beginning of the deep learning era. Each subsequent architectural innovation - VGG, ResNet, EfficientNet, ViT - pushed accuracy further, surpassing human performance of approximately 95% around 2015 and reaching over 99% by 2022.

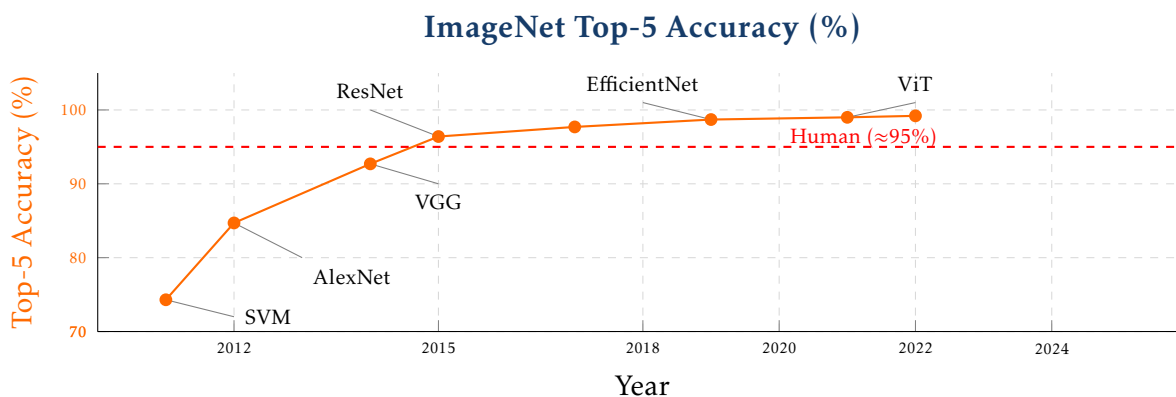


Figure 1.5: ImageNet top-5 accuracy from 2011 to 2022. Each labelled point corresponds to a key architectural innovation. The dashed red line indicates approximate human performance.

1.4.4. Community

Perhaps the most underappreciated factor is the open culture of the deep learning community. Frameworks such as PyTorch, TensorFlow, and JAX are open source, actively maintained, and freely available. The norm of releasing code alongside papers - through platforms such as Papers with Code, Kaggle, and HuggingFace - means that results are reproducible and immediately buildable upon. Research is shared on arXiv before peer review, and major conferences such as NeurIPS, ICML, and ICLR publish

proceedings openly.

This compounding effect - where each breakthrough immediately becomes everyone's foundation - has no parallel in most other scientific fields.

1.5. What Can Deep Learning Do?

Deep learning today touches almost every domain of human activity. Rather than organising applications by the models used, it is more instructive to group them by the type of task they perform.

► Session 1: What Can Deep Learning Do?

1.5.1. Language and Communication

Natural language was one of the earliest and remains one of the most impactful application areas. Machine translation systems such as DeepL and Google Translate now produce fluent output across dozens of language pairs, a task that resisted decades of rule-based approaches. Conversational agents - ChatGPT, Claude, Gemini - have brought language models to a mass audience, demonstrating capabilities in reasoning, summarisation, and question answering that were considered out of reach just a few years ago. Code generation tools such as GitHub Copilot and Cursor assist developers by completing, explaining, and generating code in real time.

1.5.2. Vision and Perception

Computer vision was the domain in which deep learning first demonstrated decisive superiority over classical methods, with AlexNet's 2012 ImageNet result marking the turning point. Today deep learning powers image classification, object detection, and semantic segmentation across a vast range of applications. Autonomous driving systems parse complex road scenes in real time, combining camera, lidar, and radar inputs to make driving decisions. Satellite and aerial imagery analysis enables large-scale monitoring of land use, deforestation, and infrastructure.

1.5.3. Healthcare and Medicine

Medical imaging has been transformed by deep learning. Systems trained on large collections of labelled scans can detect tumours, classify X-rays, and segment anatomical structures with accuracy approaching or exceeding that of specialist clinicians. Beyond imaging, deep learning is applied to genomics - identifying disease-associated variants in DNA sequences - and to clinical decision support systems that assist physicians with diagnosis and treatment planning.

1.5.4. Generation

Generative models have emerged as one of the most publicly visible applications of deep learning. Image generation systems such as Stable Diffusion and DALL-E produce photorealistic or stylised images from text descriptions. Video and music generation - tools such as Suno and Udio - extend this capability to temporal media. The same underlying technology that enables generation also enables detection: deepfake detection systems attempt to identify synthetically generated media, an increasingly important capability as generation quality improves.

1.5.5. Scientific Discovery

Perhaps the most consequential application of deep learning to date is AlphaFold **jumper2021alphafold**, DeepMind's system for predicting protein three-dimensional structure from amino acid sequence. AlphaFold solved a fifty-year-old open problem in structural biology and earned Demis Hassabis and John Jumper the 2024 Nobel Prize in Chemistry - the first Nobel Prize awarded for work driven by deep learning. Deep learning is also applied to drug discovery - predicting molecular properties and identifying candidate compounds - and to climate and weather modelling, where neural networks are beginning to challenge traditional physics-based simulation approaches.

1.5.6. Decision Making and Control

Reinforcement learning - a branch of deep learning in which agents learn by interacting with an environment - has produced some of the field's most dramatic results. AlphaGo **silver2016alphago** defeated the world champion Go player in 2016, a milestone considered a decade away by most experts at the time. Robotic systems learn manipulation and locomotion skills directly from experience, without hand-coded controllers. Recommendation systems - the algorithms that determine what appears in TikTok feeds, Netflix homepages, and Spotify playlists - are among the most economically impactful applications of deep learning, shaping the daily media consumption of billions of people.

2. CNNs in Practice

Session 9 introduced the theory and architecture of convolutional neural networks. This session focuses on how CNNs are actually used in practice. The central message is straightforward: you will almost never train a CNN from scratch. Instead, you will download a pretrained model and adapt it to your task. We examine when and how to do this, cover the practical PyTorch tools involved, and then extend beyond image classification to the three major visual tasks that CNNs also power: object detection, semantic segmentation, and instance segmentation.

► Session 10

2.1. Transfer Learning

Training a CNN from scratch on every new problem would be enormously wasteful — the low-level visual features learned by early layers are essentially the same regardless of the task. Transfer learning exploits this by reusing weights learned on one task as the starting point for another. This section covers when and how to do this effectively.

► Session 10: Three Regimes, Why Pretrained Models Work, Feature Extraction vs Fine-tuning, When to Use Which, Practical Tuning

2.1.1. Three Regimes

There are three distinct regimes for working with CNNs, each appropriate in different circumstances.

Training from scratch means initialising all weights randomly and optimising the full network on your own dataset. This requires a large labelled dataset (tens of thousands to millions of examples), significant compute (days to weeks on multiple GPUs), and is only justified when you have a genuinely novel architecture, a domain so different that pretrained weights would be useless, or when you are a large organisation with

the necessary resources. For most practitioners, training from scratch is not a realistic option.

Pretraining followed by fine-tuning means starting from a model trained on a large source dataset and then continuing training on your target dataset. This is the standard approach in research settings where moderate labelled data is available and strong performance is required. The pretrained weights provide a much better initialisation than random, allowing faster convergence and better generalisation.

Downloading and adapting is the dominant modern workflow. Pretrained models are freely available from torchvision, timm, and HuggingFace. You download a model, replace its classification head with one appropriate for your task, and either use it as a fixed feature extractor or fine-tune some layers. This approach works surprisingly well even for tasks quite different from the original pretraining, requires relatively little data and compute, and is how the vast majority of real CNN applications are built.

2.1.2. Why Pretrained Models Work

The reason transfer learning works so well is the hierarchical nature of what CNNs learn. As demonstrated by Zeiler2014, different layers of a CNN learn features at different levels of specificity:

- **Early layers** learn general low-level features: edges, colour gradients, oriented bars. These features are essentially universal — they appear in medical images, satellite imagery, microscopy, and natural photographs alike. A filter that detects a horizontal edge in an ImageNet-trained network is equally useful for detecting edges in a chest X-ray.
- **Later layers** learn increasingly task-specific features: object parts, textures associated with specific categories, combinations of lower-level features that are meaningful for the source task. These features are less transferable — a network trained to classify ImageNet categories will have late-layer features tuned for dogs, cars, and furniture, which may not be useful for classifying skin lesions.

This hierarchical structure means that pretrained weights are almost always a better starting point than random initialisation, even when the target domain differs substantially from the source. The question is not whether to use pretrained weights, but how much of the network to adapt.

Modern pretrained models are trained on increasingly large datasets: ImageNet (1.2M images, 1000 classes), LAION-5B (5.8 billion image-text pairs), and various proprietary datasets used by large companies. Models trained on larger, more diverse datasets tend to transfer better across a wider range of target tasks.

2.1.3. Feature Extraction vs Fine-tuning

Once you have a pretrained model, there are two main strategies for adapting it to a new task.

Feature extraction treats the pretrained network as a fixed feature extractor. All pretrained layers are frozen — their weights are not updated during training — and only a new classification head is trained. The pretrained network maps each input image to a feature vector, and the head learns to classify these vectors for the new task. Feature extraction is appropriate when the target dataset is small (hundreds to a few thousand examples), when compute is limited, or when the target domain is similar enough to the source that the pretrained features are directly useful.

Fine-tuning unfreezes some or all of the pretrained layers and trains them alongside the new head, typically with a much smaller learning rate. This allows the network to adapt its pretrained features to the specific characteristics of the target domain, at the cost of needing more data and compute. Fine-tuning is appropriate when the target dataset is large enough to avoid overfitting the deeper layers, or when the target domain is sufficiently different from the source that direct feature transfer is insufficient.

Catastrophic forgetting is a key risk in fine-tuning: if the learning rate is too large, gradient updates on the new task will overwrite the pretrained weights, destroying the useful features learned during pretraining. The network “forgets” what it learned before. The standard mitigation is to use a learning rate $10\times$ – $100\times$ smaller than the original pretraining learning rate when fine-tuning pretrained layers.

2.1.4. When to Use Which

The choice between feature extraction and fine-tuning depends on two factors: the size of the target dataset and the similarity between the source and target domains.

- **Small dataset, similar domain:** feature extraction. The pretrained features are directly useful, and the small dataset is not large enough to fine-tune without overfitting.
- **Small dataset, different domain:** feature extraction, but experiment with which layers to use as features. Early layers (more general) may transfer better than late layers (more specific to the source task).
- **Large dataset, similar domain:** fine-tune all layers with a small learning rate. Enough data to adapt the network without overfitting.
- **Large dataset, different domain:** fine-tune all layers, possibly for more epochs and with more careful hyperparameter tuning.

A practical rule of thumb: always start with feature extraction. It is fast, safe, and often surprisingly competitive. Add fine-tuning only if feature extraction is clearly insufficient, and proceed gradually.

2.1.5. Practical Tuning

Several practical considerations arise when adapting pretrained models.

Freezing strategy. Start with all pretrained layers frozen and train only the new head. If performance is insufficient, unfreeze layers progressively from the top — that is, unfreeze the later layers first, since they are the most task-specific and most in need of adaptation. Earlier layers, which contain more general features, should be frozen longer. This progressive unfreezing avoids catastrophic forgetting in the early stages of adaptation.

Discriminative learning rates. When fine-tuning multiple layer groups, it is common to use different learning rates for different parts of the network — a technique called discriminative learning rates. Later layers, which need more adaptation, receive a larger learning rate; earlier layers, which are already well-trained, receive a smaller one. A typical setup might use $1e-3$ for the new head, $1e-5$ for the last pretrained block, and $1e-6$ for earlier blocks. In PyTorch, this is implemented by passing parameter groups with per-group learning rates to the optimiser.

Learning rate warmup. When fine-tuning, it can help to start with a very small learning rate and increase it gradually over the first few epochs. At the start of fine-tuning, the new head is randomly initialised and may produce large, noisy gradients.

Warmup gives the head time to stabilise before the pretrained layers are significantly updated.

Data augmentation. More aggressive augmentation helps when the target dataset is small. Standard augmentations — random crops, horizontal flips, colour jitter — reduce overfitting and improve generalisation. Domain-specific augmentations can be added on top.

Early stopping. Fine-tuning can overfit quickly on small datasets. Monitor validation loss carefully and stop when it begins to rise. The model at the best validation loss should be saved and used, not the model at the end of training.

2.2. Practical CNN Training

2.2.1. Sources of Pretrained Models

Three main sources of pretrained models are available for PyTorch users.

torchvision.models is the official PyTorch model zoo, providing pretrained versions of standard architectures including ResNet, VGG, EfficientNet, and Vision Transformers. The API is straightforward:

```
from torchvision.models import resnet50, ResNet50_Weights
model = resnet50(weights=ResNet50_Weights.DEFAULT)
```

timm (PyTorch Image Models) is a community-maintained library with over 700 pretrained models, including many state-of-the-art architectures not available in torchvision. It has become the standard resource for vision research:

```
import timm
model = timm.create_model('resnet50', pretrained=True)
```

HuggingFace hosts pretrained vision-language models including CLIP, DINO, and Vision Transformers via the transformers library. It is particularly useful for models trained with self-supervised or contrastive objectives on large-scale data.

Each source provides models with documented pretraining configurations, including the normalisation statistics used during training — important for preprocessing (see below).

2.2.2. Adapting a Pretrained Model

The standard adaptation workflow in PyTorch is:

```
model = resnet50(weights=ResNet50_Weights.DEFAULT)

# freeze all pretrained layers
for param in model.parameters():
    param.requires_grad = False

# replace FC head with new classifier
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, num_classes)

# train only the new head
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

» Session 10:
torchvision and
timm, Adapting
Pretrained Model,
Image Transforms
Pipeline

Note that replacing `model.fc` automatically makes the new layer's parameters trainable (since they are freshly initialised), while all other parameters remain frozen.

For fine-tuning with discriminative learning rates:

```
# unfreeze last ResNet block
for param in model.layer4.parameters():
    param.requires_grad = True

# different learning rates per layer group
optimizer = torch.optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 1e-5},
    {'params': model.fc.parameters(), 'lr': 1e-3},
])
```

All the basics from earlier sessions still apply: `model.train()` and `model.eval()` for mode switching, `optimizer.zero_grad()` before each backward pass, and the standard training loop.

2.2.3. Image Transforms Pipeline

Images must be preprocessed before being fed to a CNN. PyTorch provides the `torchvision.transforms` module for this. A standard pipeline consists of:

```
from torchvision import transforms

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224),      # random crop and resize
    transforms.RandomHorizontalFlip(),      # data augmentation
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),                  # convert to tensor, scale to [0,1]
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225])          # normalise
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),             # deterministic crop
    transforms.ToTensor(),
    transforms.Normalize(...)
])
```

Two important points. First, **no augmentation at validation or test time** — the validation transform uses only deterministic operations (resize and center crop) to ensure consistent, reproducible evaluation. Applying random augmentations at validation time would make the loss noisy and unreliable as a stopping criterion.

Second, **normalisation statistics must match the pretraining dataset**. The values `[0.485, 0.456, 0.406]` and `[0.229, 0.224, 0.225]` are the per-channel mean and standard deviation of ImageNet. If the model was pretrained on a different dataset,

different statistics apply. Using wrong normalisation values shifts the input distribution away from what the network was trained on, degrading performance. Both torchvision and timm document the correct statistics for each pretrained model.

2.3. Beyond Classification

Image classification assigns a single label to an entire image. Many real-world applications require richer outputs: locating objects, labelling every pixel, or distinguishing individual instances of the same class. Figure 2.1 illustrates the four main visual tasks.



Figure 2.1: Four main visual tasks. Classification assigns one label to the whole image. Object detection localises and classifies multiple objects with bounding boxes. Semantic segmentation assigns a class label to every pixel. Instance segmentation additionally distinguishes individual objects of the same class.

All four tasks share a common pattern: a pretrained CNN backbone extracts features, and a task-specific head produces the required output. This is why the transfer learning principles from Section 1 apply directly — the backbone can be pretrained on ImageNet and fine-tuned for any of these tasks.

2.3.1. Object Detection

Object detection requires the network to output, for each object in the image, a bounding box (four coordinates), a class label, and a confidence score. Unlike classification, multiple objects of different classes can be present simultaneously.

Two-stage detectors. The R-CNN family of detectors (**Ren2015**) uses a two-stage approach. In the first stage, a **Region Proposal Network (RPN)** scans the feature map and proposes candidate regions that might contain objects, using anchor boxes of different scales and aspect ratios at each spatial location. In the second stage, each proposed region is extracted, pooled to a fixed size (RoI pooling), and passed through a classifier and bounding box regressor. Faster R-CNN introduced the RPN, making the whole pipeline end-to-end trainable and reducing inference time to around 0.2 seconds per image. Two-stage detectors generally achieve higher accuracy but are slower than one-stage approaches.

One-stage detectors. YOLO (**Redmon2016**) (You Only Look Once) and SSD predict bounding boxes and class probabilities directly from the feature map in a single forward pass. YOLO divides the image into an $S \times S$ grid; each cell predicts B bounding boxes with confidence scores and C class probabilities, producing an $S \times S \times (B \times 5 + C)$ output tensor. This single-pass design makes YOLO extremely fast and suitable for real-time applications. Modern YOLO versions (now at v10/v11) have largely closed the accuracy gap with two-stage detectors while maintaining high speed.

Evaluation. Object detection is evaluated using **Intersection over Union (IoU)** and **mean Average Precision (mAP)**.

IoU measures the overlap between a predicted bounding box and the ground truth box (Figure 2.2):

$$\text{IoU} = \frac{|\text{intersection}|}{|\text{union}|} \in [0, 1].$$

A prediction is counted as correct if its IoU with the ground truth exceeds a threshold, typically 0.5.

Average Precision (AP) for a single class summarises the precision-recall tradeoff: as the confidence threshold is varied, different numbers of predictions are accepted, tracing out a precision-recall curve. AP is the area under this curve. Mean AP (mAP) averages AP across all classes. The COCO benchmark uses a stricter standard: mAP averaged over IoU thresholds from 0.5 to 0.95 in steps of 0.05.

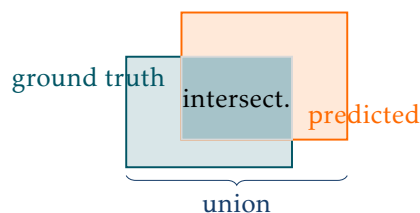


Figure 2.2: IoU measures the overlap between predicted and ground truth bounding boxes. A prediction is counted as correct if IoU exceeds a threshold.

2.3.2. Semantic Segmentation

Semantic segmentation assigns a class label to every pixel in the image, producing an output label map of the same spatial size as the input. All pixels belonging to the same class receive the same label, regardless of whether they belong to different object instances.

The dominant architecture for semantic segmentation is the **encoder-decoder**. The encoder is a CNN backbone that progressively reduces spatial resolution while extracting rich features. The decoder upsamples the low-resolution feature maps back to the input resolution, recovering spatial detail. **Skip connections** between corresponding encoder and decoder layers concatenate high-resolution encoder features with upsampled decoder features, helping the decoder recover fine spatial detail that was lost during downsampling.

U-Net (Ronneberger2015) introduced this architecture for medical image segmentation and remains widely used. **DeepLab v3 (Chen2017)** uses atrous (dilated) convolutions to capture multi-scale context without reducing resolution. **SegFormer (Xie2021)** replaces the CNN backbone with a Vision Transformer, achieving state-of-the-art results on standard benchmarks.

Semantic segmentation is evaluated using **mean Intersection over Union (mIoU)**: IoU is computed per class (comparing predicted and ground truth pixel masks for each class) and averaged across all classes.

2.3.3. Instance Segmentation

Instance segmentation goes beyond semantic segmentation by distinguishing individual object instances. Two people standing next to each other receive different instance masks even though they belong to the same class. The output is a set of per-instance binary masks, each paired with a class label and confidence score.

Mask R-CNN (He2017) extends Faster R-CNN with an additional mask prediction head. For each detected bounding box, a small FCN predicts a binary mask for that instance. RoI Align (an improvement over RoI pooling that preserves spatial alignment)

extracts fixed-size features from each proposed region, which are then passed to the mask head. Mask R-CNN is the standard baseline for instance segmentation.

SAM (Segment Anything Model) (Kirillov2023) is a foundation model trained on 1 billion masks. Given a prompt (a point, box, or text), SAM segments the corresponding object in zero-shot — without task-specific fine-tuning. SAM has become an important tool not only for inference but also for annotation assistance, dramatically reducing the cost of creating instance segmentation datasets.

Panoptic segmentation combines semantic and instance segmentation into a unified output: background regions (sky, road, grass) receive semantic labels while foreground objects receive instance masks. It has become the modern standard for comprehensive scene understanding.

Instance segmentation is evaluated using mAP computed on masks rather than bounding boxes — IoU is measured between predicted and ground truth pixel masks.

2.3.4. Annotation Cost

A practical consideration often underestimated by practitioners is annotation cost. The annotation requirements differ dramatically across tasks:

Task	Annotation type	Time per image	Example dataset
Classification	Image-level label	Seconds	ImageNet
Detection	Bounding boxes	Minutes	COCO
Semantic segmentation	Pixel-wise labels	30–90 min	Cityscapes
Instance segmentation	Per-instance masks	Hours	COCO

Table 1: Annotation cost comparison across visual tasks.

Annotation cost is frequently the real bottleneck in building vision systems — not model complexity or compute. A state-of-the-art instance segmentation model is of little value if you cannot afford to annotate enough training data. Several approaches help reduce this burden:

SAM for annotation assistance — SAM can generate high-quality segmentation masks interactively, with a human providing only a point or bounding box prompt. This can reduce mask annotation time by an order of magnitude.

Semi-supervised and weakly supervised methods — train models using a small amount of expensive annotations combined with a large amount of cheap annotations (e.g. image-level labels for a detection model). Many recent methods exploit this tradeoff effectively.

Synthetic data — render annotated training data automatically from 3D simulations or procedural generation. Particularly useful for specialised domains where real annotations are prohibitively expensive (autonomous driving, robotics).

Acknowledgements

These lecture notes were prepared with the assistance of Claude, a large language model developed by Anthropic. All content has been reviewed, edited, and approved by the author.